



SYSTEM DESIGN & TECHNICAL OVERVIEW

Kali Tools GUI

מסמך תכן והסבר מערכת — ארכיטקטורה, סוכנים וזרימת נתונים

מחבר: **הראלי דודאי**

ת.ז.: 027225721

דוא"ל: hareli26@gmail.com

מוסד: **מכללת HackerU** · קורס: אדריכלות AI

מנחה: שחף

תאריך: יולי 2026

מאגר קוד: github.com/Hareli26/kali-tools-gui

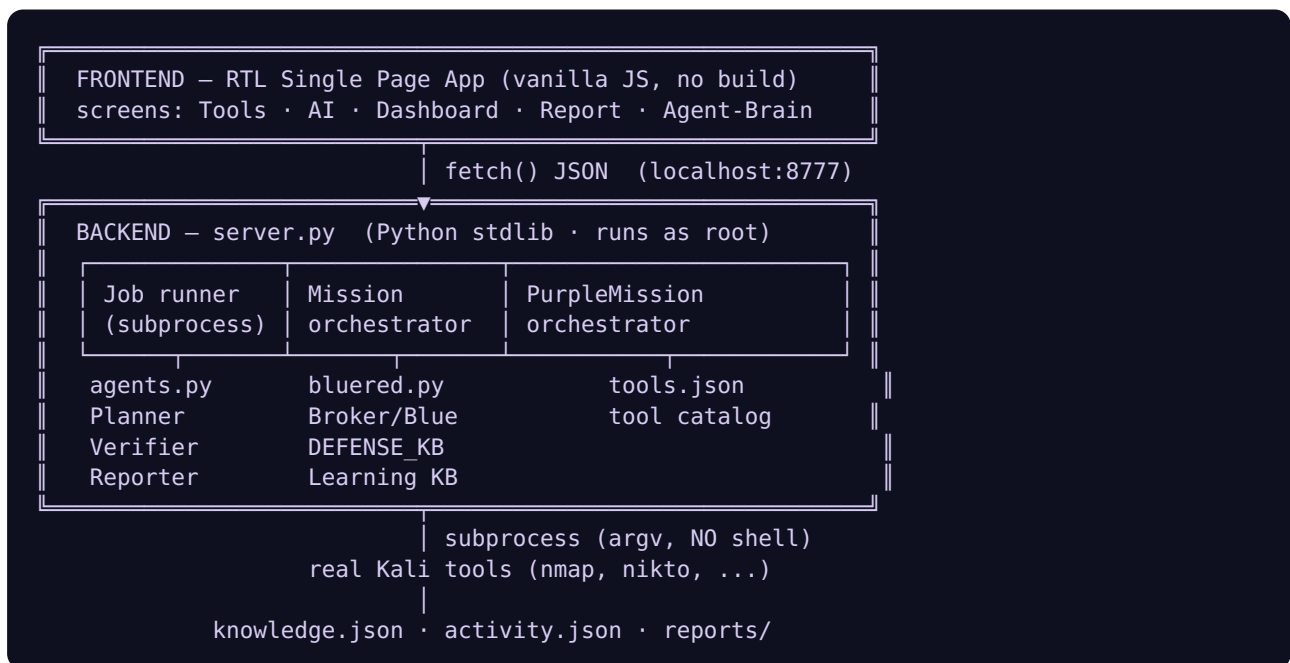
תוכן עניינים

1. מבוא
2. סקירת ארכיטקטורה כללית
3. שכבת ה-Backend (server.py)
4. מנוע הכלים ובניית פקודות
5. שכבת הסוכנים (agents.py)
6. שכבת Purple Team (bluered.py)
7. זרימת נתונים — מאיפה הסוכנים יודעים?
8. שכבת ה-Frontend ותצוגת המוח
9. אבטחה
10. פריסה ותפעול בייצור
11. טכנולוגיות (Tech Stack)
12. צילומי מסך מהמערכת
13. סיכום

1. מבוא

מסמך זה מתאר את התכן ההנדסי של מערכת **Kali Tools GUI**: ממשק גרפי ושכבת סוכנים בגישת Purple Team המורצת מעל כלי Kali Linux תחת WSL2. המסמך נועד להסביר כיצד המערכת בנויה, כיצד רכיביה משתלבים, ובמיוחד — מאיפה הסוכנים "יודעים" לפעול ולקבל נתונים.

2. סקירת ארכיטקטורה כללית



איור 1 — שכבות המערכת וזרימת הבקרה.

3. שכבת ה-Backend (server.py)

שרת HTTP רב-תהליכי (ThreadingHTTPServer) בספריית פייתון הסטנדרטית. מגיש את קבצי ה-Frontend וחושף API ב-JSON.

3.1 מודל ה-Job

כל הרצת כלי היא אובייקט Job : מריץ subprocess.Popen בקבוצת תהליכים נפרדת, קורא פלט בתהליכון (thread) עם מגבלת גודל, ומאפשר עצירה נקייה. הפלט זמין ל-polling מהדפדפן.

3.2 נקודות קצה עיקריות

נקודת קצה	תפקיד
GET /api/tools	קטלוג הכלים + סטטוס התקנה
POST /api/run	הרצת כלי בודד
POST /api/plan	Planner: כוונה+מטרה → תוכנית שלבים
POST /api/mission	הרצת משימה (Executor+Verifier+Reporter)
POST /api/purple	הרצת Purple (Red→Broker→Blue→Orchestrator)
GET /api/dashboard	מצב סוכנים חי + מודיעין + יומן
GET /api/brain	גרף יכולות הסוכן + נתונים חיים
GET /api/active	המשימה הפעילה + התקדמות חיה (מפעיל את סרגל הריצה הגלובלי)
GET /api/vault/graph	גרף כספת Obsidian (דוחות·איומים·MOC) לתצוגת Graph View-ה
POST /api/obsidian/export	ייצוא הכספת (Markdown + frontmatter + wikilinks)
POST /api/fix · GET /api/fix/<sig>	תוכנית תיקון · יישום (דורש confirm)
POST /api/users/ · GET /api/users add remove	ניהול allowlist — אדמין בלבד (403 אחרת)
GET /api/whoami	זהות המשתמש + האם אדמין
GET /api/health	בריאות השירות (גרסה, uptime)

4. מנוע הכלים ובניית פקודות

הקטלוג `tools.json` מגדיר לכל כלי: בינארי, קטגוריה, ואת שדות הטופס (סוג, דגל, ברירת מחדל). פונקציית `build_argv` בונה את הפקודה כמערך `argv` — כל אסימון בנפרד, ללא `shell`. זהו עיקרון אבטחה מרכזי: ערכי המשתמש לעולם אינם מתפרשים ע"י מעטפת, ולכן הזרקת פקודות נמנעת.

```
argv = [binary]
for opt in options:          # לפי סדר ההגדרה
    if bool ומסומן: argv += [flag]
    elif ערך קיים: argv += [flag, value] # או "flag=value"
argv += [positional target]
```

5. שכבת הסוכנים (agents.py)

מנוע מבוסס-חוקים (Playbooks) — ללא תלות ב-LLM, דטרמיניסטי וניתן לביקורת.

Planner (מתכנן)

מקבל כוונה חופשית + מטרה. מנקד כל Playbook לפי מילות מפתח (עברית+אנגלית), בוחר את המתאים ובונה שלבים (כלי + פקודה + הסבר + הצעה). לדוגמה: "בדיקת dig/dnsenum → /DNS" /dnsrecon.

Executor (מבצע)

מריץ כל שלב ברצף כ-Job, עם מגבלת זמן לכל שלב (ברירת מחדל 300 שניות) כדי שכלי איטי לא יתקע את המשימה.

Verifier (מאמת)

מנתח את הפלט וקוד היציאה של הכלי. באמצעות דפוסי `regex` מזהה שגיאות (דרוש `root`, מטרה לא זמינה) ומחלץ ממצאים (פורטים פתוחים, CVE, חולשות). מפיץ verdict: תקין / ממצאים / אזהרה / נכשל.

Reporter (מדווח)

מרכיב דוח Markdown: סיכום מנהלים, ממצאים עיקריים ופירוט שלבים. אם מוגדר Ollama — משדרג את הניסוח ב-LLM.

6. שכבת Purple Team (bluered.py)

Broker (מתווך) 🟢

מקבל את ממצאי הצוות האדום, ממפה כל ממצא לכלל הגנה מתוך DEFENSE_KB, מאחד עדויות ומדרג לפי חומרה.

Blue Team (צוות כחול) 🔵

בסיס ידע של ~15 כללי הגנה. לכל איום: תיאור, פעולות הגנה/הקשחה, זיהוי וניטור, תצורה לדוגמה, ומיפוי MITRE ATT&CK (למשל T1190, T1110).

Learning (למידה) 🧠

כל ממצא מנורמל לחתימה ונצבר ב- knowledge.json (מונה + first/last seen). כך המערכת "לומדת" אילו איומים חוזרים לאורך זמן.

Orchestrator (מתזמר) 🟣

מנצח את כל הזרימה: מריץ את הצוות האדום, מפעיל את המתווך והכחול, מעדכן את הלמידה, ומפיק דוח Purple מלא.

Remediation (סוכן מתקן) 🔧

ממפה כל חתימת איום לתיקון מעשי (למשל fail2ban, עדכוני אבטחה) ומיישם אותו על המערכת — רק לאחר אישור מפורש, עם גיבוי קונפיגים ורישום ביומן ביקורת. תיקונים מסוכנים (SSH/firewall) מסומנים כ"ידני" ואינם רצים אוטומטית, למניעת נעילה.

7. זרימת נתונים — מאיפה הסוכנים יודעים?

שאלת מפתח: הסוכנים אינם "ממציאים" נתונים. הם שואבים משלושה מקורות ברורים וניתנים לביקורת.

מקור	מה הוא מכיל	מי משתמש בו
1. ידע סטטי (קוד/JSON)	tools.json (קטלוג ופקודות), agents.py (+ Playbooks) דפוסי חילוץ, bluered.py (כללי הגנה + MITRE)	Planner, Verifier Blue
2. נתונים אמיתיים (הכלים)	הפלט האמיתי של כלי Kali על המטרה — פורטים, גרסאות, חולשות. אמת הקרקע.	→ Executor Verifier
3. זיכרון מצטבר (קבצים)	knowledge.json (למידה), activity.json (יומן), reports/ (דוחות)	Learning Dashboard, Brain

```

intent + target
  |
  v
[Planner] — reads tools.json + PLAYBOOKS → plan (steps)
  |
  v
[Executor] — subprocess → REAL Kali tool → raw output
  |
  v
[Verifier] — regex patterns → findings
  |
  v
[Broker] — maps to DEFENSE_KB → threats + defenses
  |
  v
[Learning] — writes → knowledge.json (grows every run)
  |
  v
[Reporter] → report.md → reports/ + activity.json
  
```

איור 2 — זרימת הנתונים בין הסוכנים. הממצאים האמיתיים מגיעים תמיד מהכלים עצמם.

8. שכבת ה-Frontend ותצוגת המוח

אפליקציית עמוד יחיד (SPA) ב-vanilla JS ללא שלב build. המצבים: כלים, עוזר AI, מרכז בקרה, תצוגת Obsidian, וניהול משתמשים (אדמין). הדשבורד מציג את מצב הסוכנים החי (נגזר מהרצות פעילות), מודיעין נצבר ויומן פעילות.

8.0 סרגל ריצה גלובלי + תצוגת Obsidian + ניהול משתמשים

סרגל ריצה גלובלי: רכיב קבוע הסוקר את `/api/active` ומופיע בכל מסך כל עוד משימה פעילה — מציג את הכלי הנוכחי, ההתקדמות (שלב X/Y) והפאזה, עם חזרה בלחיצה. מכיוון שהמשימות רצות כ-threads בשרת, הן אינן נעצרות בניווט, והסרגל אף מתחבר מחדש לאחר רענון דף. **תצוגת Obsidian:** גרף force-directed מבוסס-SVG הנבנה מ-`/api/vault/graph` (צמתי דוחות/איומים/MOC, קשתות wikilink) עם גרירה, זום/הזזה, חיפוש ופתיחת פתק. **ניהול משתמשים:** מסך אדמין (מוצג לפי `/api/whoami`) לעריכת ה-allowlist דרך `*/api/users` — הכתיבה לקובץ אטומית ו-oauth2-proxy טוען אותו חיה.

8.1 תצוגת "מוח סוכן"

לחיצה על סוכן פותחת ויזואליזציה עתידנית מבוססת SVG: ליבה זוהרת במרכז, יכולות במסלולים סביבה, סינפסות עם חלקיקי אנרגיה, וליבה "נושמת". המנגנון המרכזי הוא **Semantic Zoom (Level-of-Detail)**: ככל שמתקרבים (גלגלת) נחשפות שכבות עמוקות יותר — יכולות → כלים → פרטים. הנתונים נשאבים מ-`brain.json` ומועשרים בנתונים חיים (כלים מותקנים, ממצאים ידועים) דרך `/api/brain`.

9. אבטחה

- **בניית פקודות בטוחה:** מערך `argv` ללא `shell` — מונע הזרקת פקודות.
- **האזנה מקומית:** ברירת מחדל `127.0.0.1` בלבד — התהליך (`root`) אינו חשוף לרשת.
- **אימות אופציונלי:** טוקן משותף (`KALIGUI_TOKEN`) כהגנה נוספת.
- **מניעת traversal:** הגשת קבצים סטטיים מוגבלת לתיקיית `web` בלבד.
- **שלב אישור:** שכבת ה-AI מציעה תוכנית; ההרצה (וכל תיקון) מתבצעת רק לאחר אישור המשתמש.
- **אימות בענן:** התחברות Google + רשימת מיילים מאושרים (`oauth2-proxy`) לגישה מבחוץ.
- **ניהול משתמשים מוגן:** נקודות `*/api/users` פתוחות **לאדמין בלבד** (403 אחרת); אימות תקינות אימייל ומניעת נעילה עצמית (אי אפשר להסיר את האדמין).
- **ביקורת:** כל פעולה (הרצה-תיקון-התקנה-הוספת/הסרת משתמש) נרשמת עם זהות המשתמש שביצע אותה.

10. פריסה ותפעול בייצור

המערכת מותקנת כשירות **systemd** בתוך Kali WSL (`kali-gui.service`) עם הפעלה אוטומטית והתאוששות מקריסה (`Restart=on-failure`). מתקין חד-פעמי (`setup.ps1` / `Install.bat`) מבצע: בדיקת דרישות, תיקון DNS, התקנת השירות, ורישום משימת Windows להעלאת WSL בהתחברות.

```
systemctl status kali-gui      # מצב
systemctl restart kali-gui     # הפעלה מחדש
journalctl -u kali-gui -n 50   # לוגים
```

עמידות: טיפול גלובלי בשגיאות בכל בקשה, גיזום היסטוריית הרצות בזיכרון, לוגים לקובץ ול-journald, ושמירת דוחות לדיסק (שורדים אתחול).

10.1 פריסה בענן מאובטחת (VPS)

לגישה מבחוץ, האפליקציה נשאר על 127.0.0.1 ומעליה שרשרת הגנה:

```
Internet —HTTPS(443)—> Caddy —> oauth2-proxy —> app (127.0.0.1:8777)
                        TLS auto (Let's Encrypt)   Google login
                                                    + email allowlist
```

ה-**oauth2-proxy** אוסף **התחברות Google** ומאמת מול רשימת מיילים מאושרים (האדמין מוסיף/מסיר משתמשים בקובץ). המשתמש המאומת מועבר לאפליקציה בכותרת `X-Forwarded-Email`. **לוג ביקורת תלת-שכבתי:** Caddy (גישות) · `oauth2-proxy` (התחברויות) · האפליקציה (`audit.log`) — מי הריץ/תיקן מה). מתקין אוטומטי (`install-vps.sh`) מקים את הכל כולל `firewall` ו-`fail2ban`, וקיימת גם חלופת `docker-compose`.

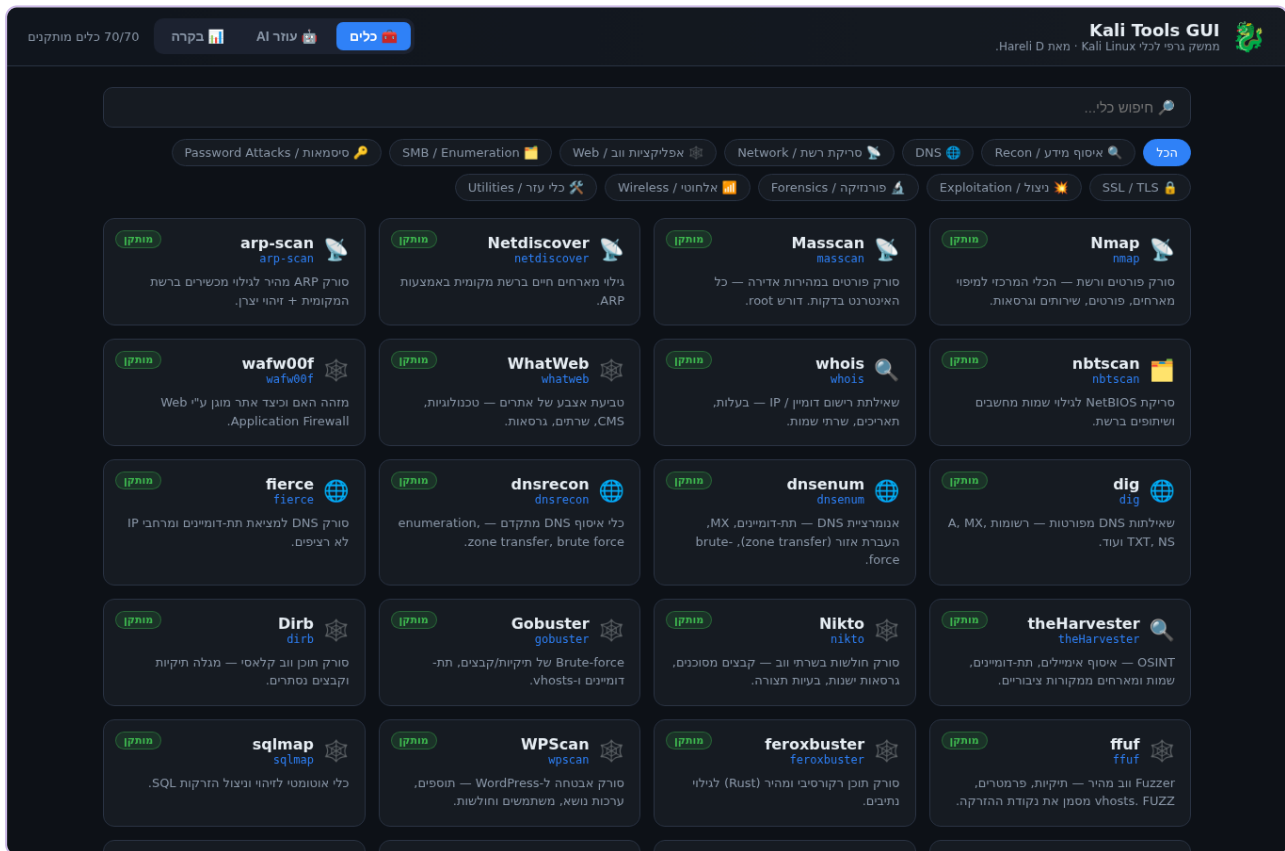
10.2 שרת MCP

רכיב `mcp/kali_gui_mcp.py` חושף את ה-REST API ככלי **MCP** — כך ש-Claude (או כל לקוח MCP) יכול להפעיל את המערכת בשיחה: להריץ כלים, לתכנן ולהריץ בדיקות ו-Purple, לקרוא מודיעין, ולהחיל תיקונים (עם אישור). הוא client דק מעל ה-API; הליבה נשארת ללא תלויות.

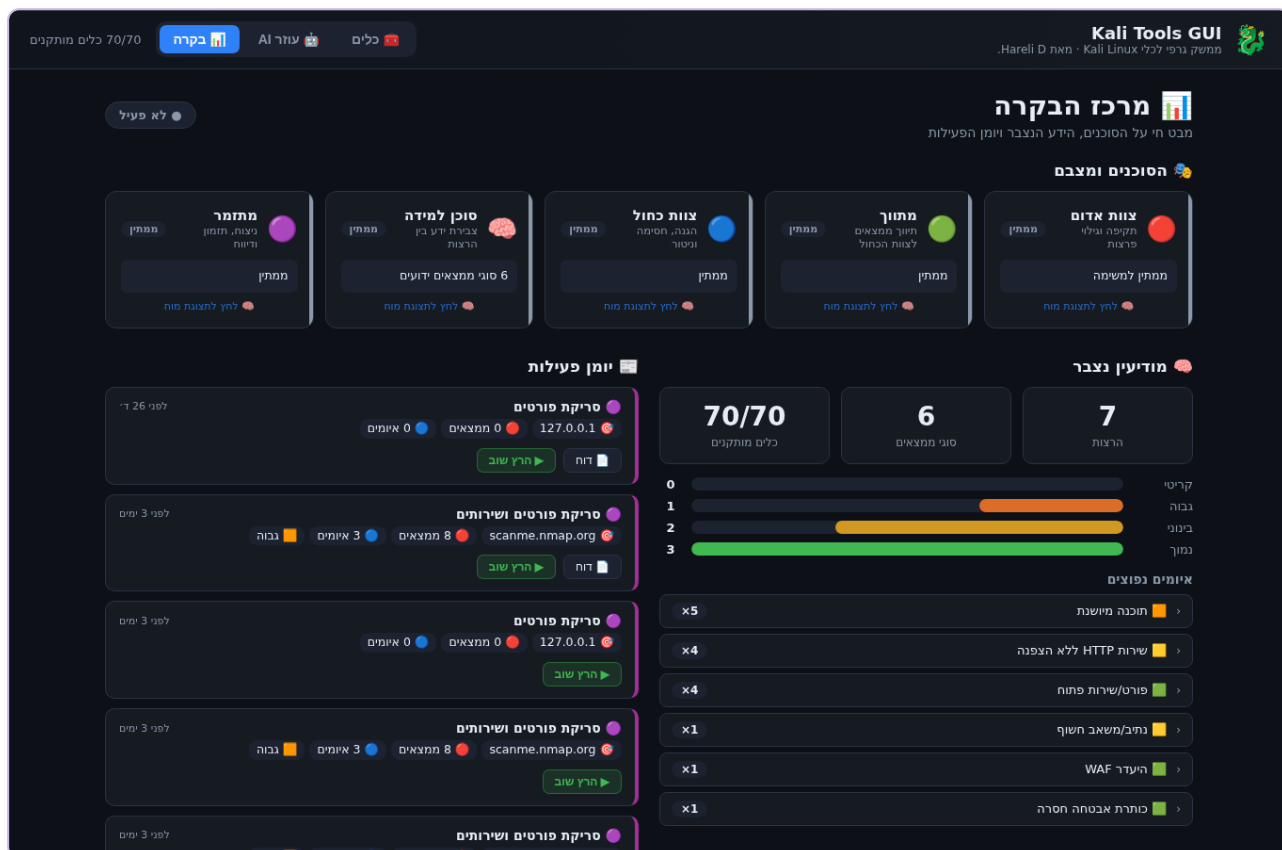
11. טכנולוגיות (Tech Stack)

שכבה	טכנולוגיה	נימוק
Backend	Python 3 — ספרייה סטנדרטית בלבד	אפס תלויות; עובד על Kali מינימלי
Frontend	HTML/CSS/Vanilla JS (ללא build)	פשטות, אפס כלי בנייה, RTL
ויזואליזציה	SVG + requestAnimationFrame	גרפיקה וקטורית אינטראקטיבית + אנימציה
אחסון	SQLite (db.py) + ייצוא Obsidian	עמידות, שאילתות, הגירה אוטומטית מ-JSON
סביבת ריצה	Kali Linux תחת WSL2	גישה מלאה לכלי הפנטסט
שירות	systemd + Windows Task Scheduler	זמינות תמידית

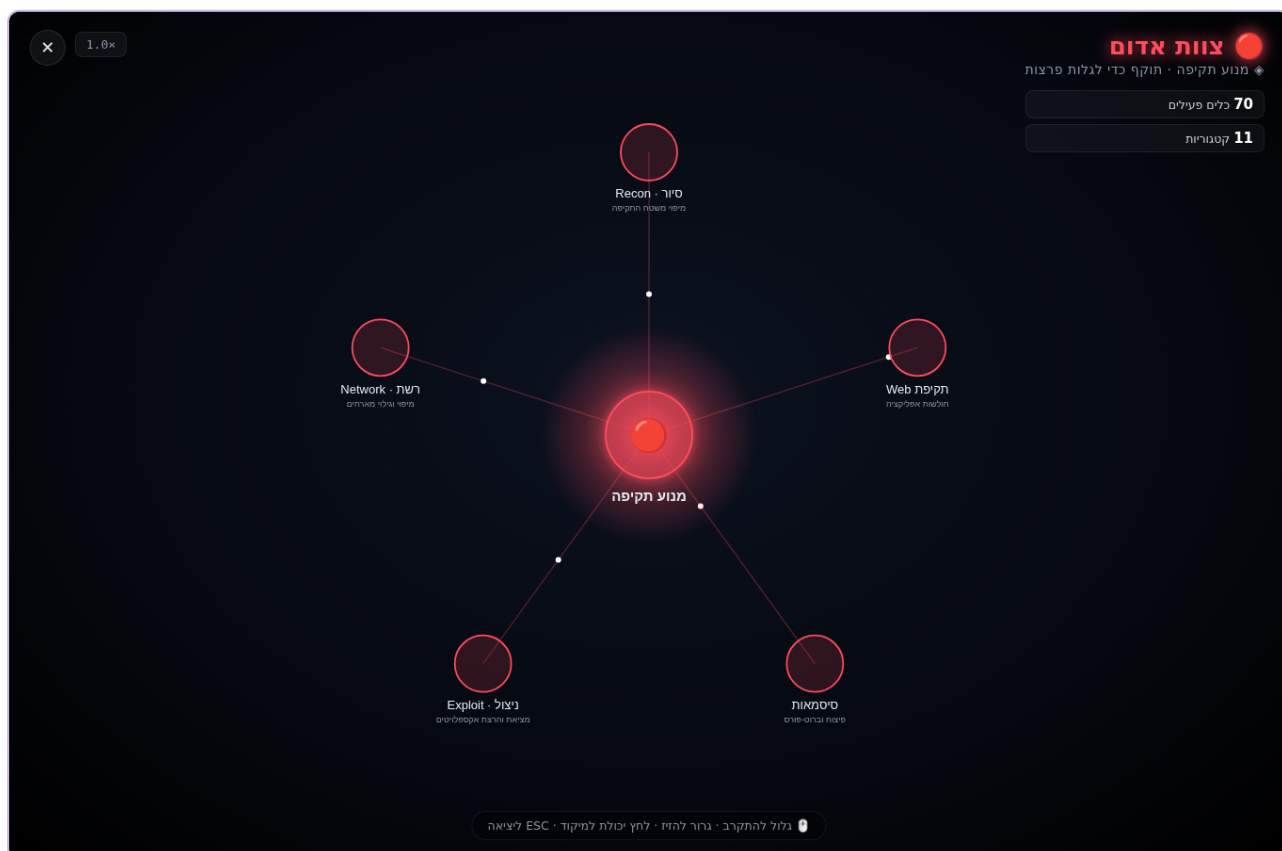
12. צילומי מסך מהמערכת



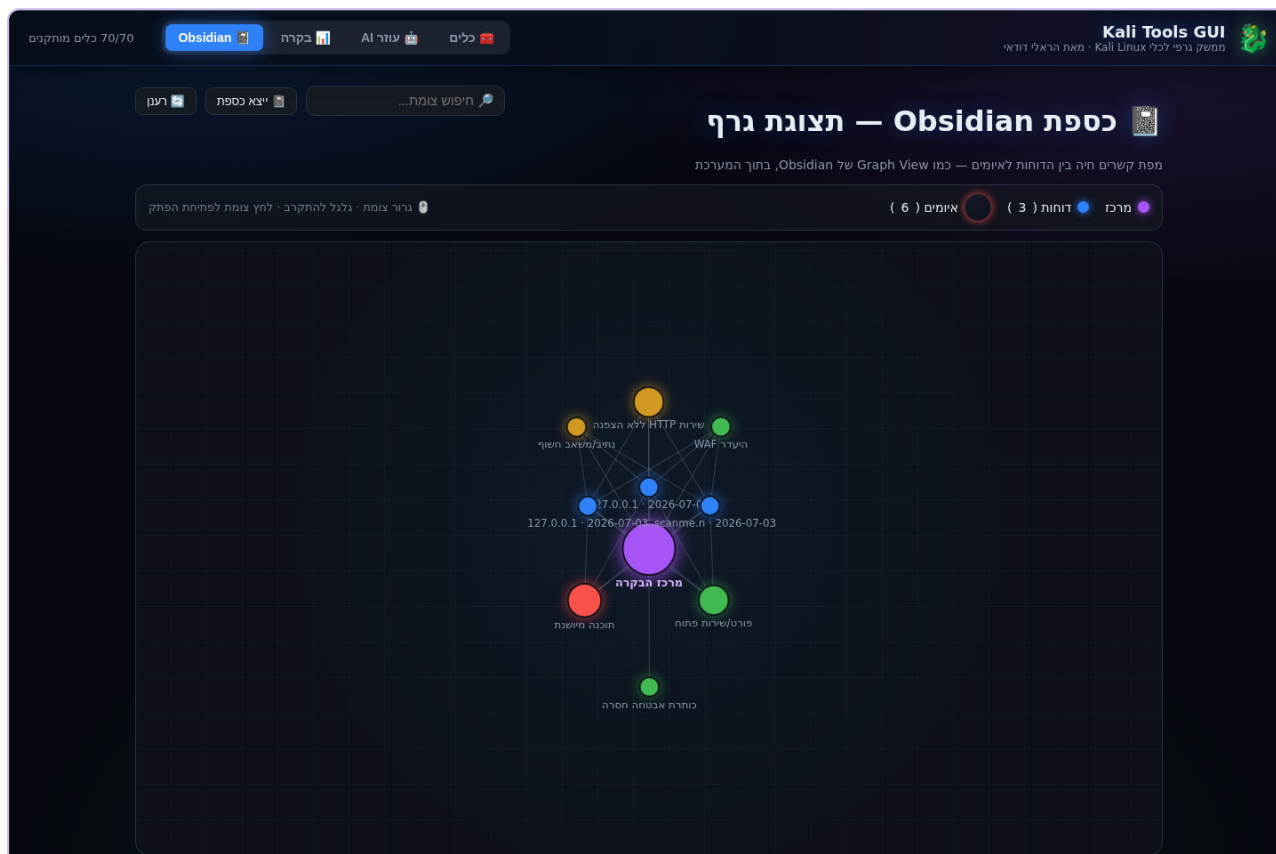
מסך 1 — מצב "כלים": קטלוג 72 הכלים עם חיפוש, סינון לפי קטגוריה וזיהוי מותקן.



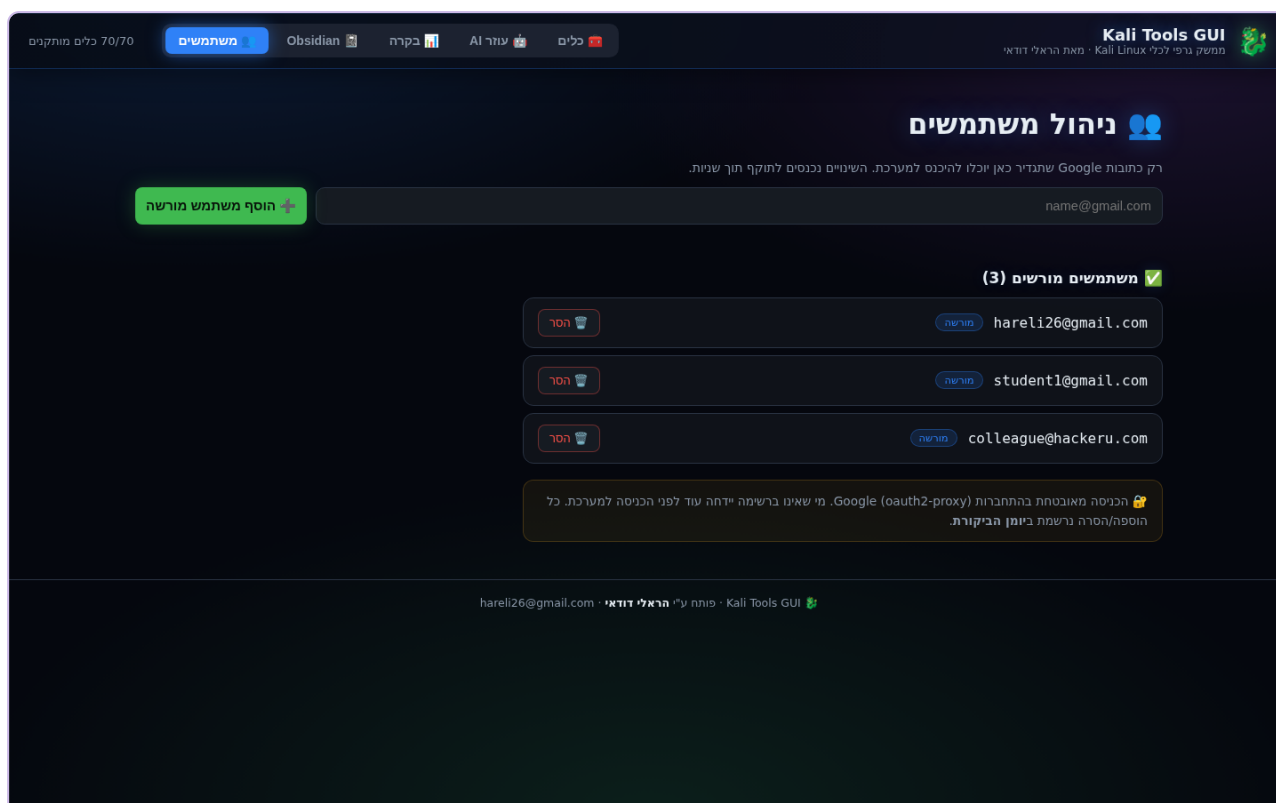
מסך 2 — מרכז הבקרה: מצב הסוכנים החי, מודיעין נצבר (חומרה, איומים נפוצים) ויומן פעילות.



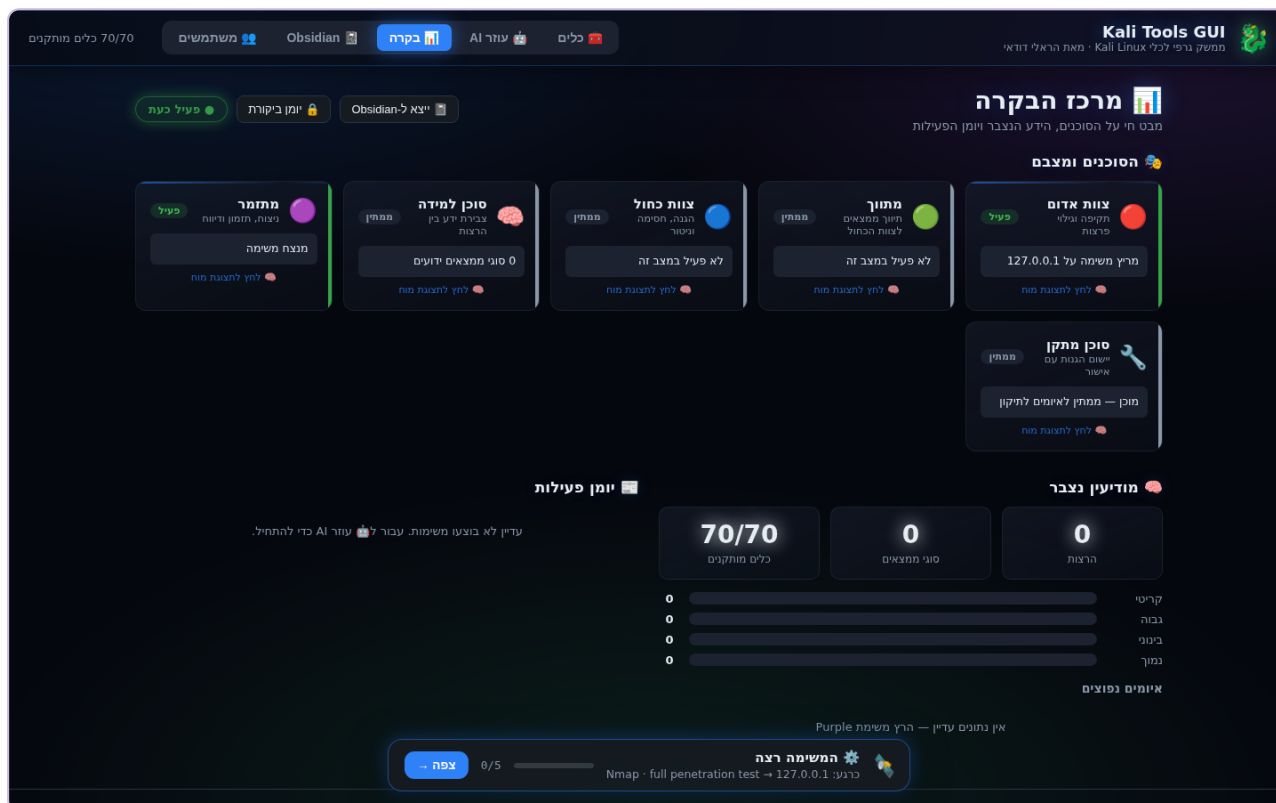
מסך 3 — תצוגת "מוח סוכן" (הצוות האדום): ליבה זוהרת, יכולות במסלולים, סינפסות ונתונים חיים.



מסך 4 — תצוגת גרף Obsidian מובנית: מפת קשרים בין דוחות (כחול) לאיומים (צבועים לפי חומרה) סביב צומת מרכזי.



מסך 5 — ניהול משתמשים (אדמין): הוספה/הסרה של חשבונות Google מורשים, עם מניעת נעילה עצמית ורישום ביומן.



מסך 6 — סרגל הריצה הגלובלי: חיווי חי של המשימה הפעילה (הכלי הנוכחי, התקדמות) הנראה בכל מסך עם חזרה בלחיצה.

13. סיכום

המערכת מדגימה ארכיטקטורה מודולרית וברורה: מנוע כלים בטוח, שכבת סוכנים מבוססת-ידע דטרמיניסטית, גישת Purple Team שמחברת בין תקיפה להגנה, ולמידה מצטברת. כל רכיב שקוף וניתן להרחבה — הוספת כלי, Playbook או כלל הגנה נעשית בעריכת נתונים בלבד. התוצאה: כלי חינוכי ומעשי המנגיש בדיקות אבטחה ומחבר אותן לפעולה הגנתית.

— סוף מסמך התכן · הראלי דודאי · מכללת 2026 · HackerU —